

云计算大作业技术文档（跨学科知识图谱智能体）

一、小组分工明细

（一）小组成员构成

学号（按序）	姓名
10234507008	刘玥萌
10235501426	娄屹宏
10235501430	甫政洋

表 1 小组成员表

（二）小组分工

模块	具体描述	主要贡献者（50%以上）	来自其他组员的优化（40%以上）
前端	index.html项目主页	娄屹宏	无
	counts_and_tokens.js动态访问统计	娄屹宏	无
	graph.html知识图谱	甫政洋	无
	app.js图谱结构	刘玥萌	核心底层逻辑：甫政洋
	知识图谱下载功能	甫政洋	无
	Echarts图表美术优化	刘玥萌	无
	find.html知识查询	甫政洋	无
	find.js知识查询运行逻辑	甫政洋	无
	navigater.css导航栏样式	娄屹宏	无
后端	页面异步路由	娄屹宏	无
	llm_graph_builder大模型路由	刘玥萌	无
	chat大模型路由	甫政洋	无
	CSS与JS静态文件挂载	娄屹宏	无
大模型	Doubao-Seed-1.8大模型api调用	刘玥萌	大模型费用与接口：娄屹宏
	大模型提示词工程	刘玥萌	无
	大模型内容质量校验	刘玥萌	无
	大模型节点剪裁与修复	刘玥萌	无
	llm_graph_builder.py图结构生成模型	刘玥萌	无
	chat.py知识查询模型	甫政洋	models.lib大模型工具库：刘玥萌
数据库	MongoDB和TinyDB数据库	娄屹宏	无
	布隆过滤器	娄屹宏	无
	阿里云平台部署	娄屹宏	无
云原生技术	Dockerfile	刘玥萌	提供模版：甫政洋
	main.py云服务器启动程序	娄屹宏	无
	requirements.txt环境配置	甫政洋	无
	README.md	甫政洋	无
文本	structure.txt	甫政洋	无
	技术文档	各自书写	无

表 2 成员分工表

本项目部署的核心在于云原生：云平台部署基于阿里云的 ECS 云服务器，将应用后端服务部署在云服务器上，借助云平台的弹性资源与稳定性，支撑系统运行；云数据库基于 MongoDB 或 TinyDB，同时包含“元数据缓存”、“图数据缓存”等功能，通过云存储的方式管理知识图谱的元数据、图数据，实现数据的云端存储与访问；Docker 基于 Dockerfile 实现容器化部署，用于封装后端服务（含功能路由、数据库运维等各个独立模块），实现应用的环境隔离、快速部

署与跨环境移植；Serverless 基于云平台自身的能力，实现后端服务的无服务器化运行，无需关注服务器资源管理，由云平台自动弹性伸缩资源，适配请求量的动态变化；微服务基于后端功能的可拆分性，各自为互不干扰的独立模块（如“功能路由”、“数据库运维”、“页面路由”等，无任何控制流和数据依赖性），各模块通过 HTTP 请求实现跨服务通信，具备独立部署、按需扩展的微服务特性。

2. 项目的功能实现所需要具备的云服务性质

本项目功能实现的核心在于保障并发性、稳定性和扩展性。

稳定性：传输层与通讯机制基于 TCP，保障数据传输的可靠性，避免请求丢失；后端采用 FastAPI+Uvicorn 的高并发异步通讯机制，这种异步架构能大幅减少请求阻塞，提升服务响应的稳定性；大模型调用层配备提示词工程、生成校验机制、节点裁剪与修复机制，抑制大模型幻觉、校验输出结果，保障知识图谱数据的准确性与服务稳定性；容器化部署（Docker）封装服务实现环境隔离，避免不同服务间的依赖冲突，同时云平台（阿里云 ECS）提供的基础资源具备高可用性，降低服务宕机风险。

扩展性：在微服务架构下，后端按功能拆分为独立微服务模块（功能路由、数据库运维、页面路由等），各模块可独立升级、按需扩展（如大模型模块需扩容时，仅调整火山引擎的调用资源即可）；在 Serverless 架构下，云平台可根据请求量自动弹性伸缩资源，无需手动扩容服务器，适配业务流量的动态变化；云数据库（MongoDB 或 TinyDB）支持云端存储扩容，满足知识图谱数据量增长的需求；在模块化功能设计下，前端功能（查询、交互、问答等）与后端路由模块均为独立单元，新增功能时可直接扩展对应模块，不影响现有服务。

并发性：异步通讯与高并发服务器（异步框架 FastAPI+异步服务器 Uvicorn）天然支持高并发请求处理，避免同步架构的线程阻塞问题；数据库运维层引入布隆过滤器，通过缓存查询结果减少数据库的重复请求，降低数据库压力，间接提升并发处理能力，降低应用运行的费用开销；阿里云 ECS 云服务器+Docker 容器化部署，可通过多容器实例水平扩展后端服务，承载更高的并发请求量。

3. 项目的业务逻辑全流程

作者将基于用户获取知识图谱的全流程，来直观地介绍整个项目的业务逻辑。

首先，服务端已通过 `Dockerfile` 构建出镜像，并基于该镜像在服务器内启动了对应的容器，同时配置好了容器 8000 端口和服务器主机 8000 端口之间的映射关系，以上的步骤保证了大模型与数据库均正确连接。

接着，用户在浏览器打开“公网地址/8000:index.html”，于是服务器端接收到了对 `index.html` 的请求，并进行异步处理反馈，将 `index.html` 以及内部引用的所有已挂载的静态文件一起异步并发地返回给用户，使得用户的浏览器可以渲染完整的 `index.html` 首页。紧接着，`index.html` 首页右下角的动态访问展示图表所关联的 `counts_and_tokens.js` 开始运行，并有序、间断地向服务器发起请求，来获得当前网站已处理的访问量和 `tokens` 量。服务器收到这些请求后，会访问对应的数据库，并将结果返回给 `counts_and_tokens.js`，从而实时动态地更新 `index.html` 首页的渲染。

其次，用户将鼠标悬浮于导航栏的“可视化大屏”，触发导航栏 `navigater.css` 的样式事件，向用户展示出“绘制知识图谱”。用户点击“绘制知识图谱”后，会向服务器发起对 `graph.html` 的页面请求，与上文对 `index.html` 的请求类似，服务器会异步地处理并反馈，从而返回与 `graph.html` 相关联的全部文件，使得用户的浏览器能渲染完整的 `graph.html` 页面。紧接着，`app.js` 会向服务器再次请求图谱数据案例文件，从而能在浏览器端渲染完整的 `graph.html` 页面与图谱案例。

然后，用户在输入框输入“最小二乘”，并点击“查询”，从而触发 `POST` 请求。服务器端会异步并发地接受该请求，随后异步并发地运行大模型的调用模块，在经历提示词工程、生成校验、节点剪裁与恢复这些机制后，会异步地获得图谱数据。服务器会立刻更新元数据，并在数据库中缓存本次查询和对应的图谱数据，同时将该次查询加入自己进程的布隆过滤器（服务器通过多进程异步并发地处理客户端请求，每个进程都有自己对应的布隆过滤器）。随后，服务器将该图谱数据返回给用户。用户的浏览器在接受到图数据后，会渲染图谱，并在用户鼠标悬浮到节点和边上时触发事件，生成文本框，从而对节点或边做进一步的具体描述。

最后，当其他用户也通过浏览器发起对“最小二乘”知识图谱的查询时，如果被该进程截获，则会触发布隆过滤器命中，从而引导该进程去数据库内查找对

应关键字的图谱数据，并返回给用户。当然，布隆过滤器会误判，如果数据库找不到对应的图谱数据，则会重新执行前面的大模型调用流程。

ps：数据库部分。在项目开发时，一直使用学校“当代数据管理系统”的 MongoDB 账号作为后端的云数据库，但是突然在 1 月 21 号晚上 10:00 以后就不能使用了。作者考虑了两种解决办法：租用阿里云平台的 MongoDB，但是费用在 600 元以上，过于昂贵；直接在 MongoDB 官网注册，但是阿里云服务器不能稳定访问 MongoDB 官网。这两种办法均不能有效解决问题。由于项目基本完成，无法快速将文档型数据库切换成 PostgreSQL 关系型数据库，并且作为云数据库不能使用 PC 端本地的 MongoDB，只能紧急使用 Python 的 TinyDB 来代替 MongoDB，从而造成两处并发性的损失：由 Uvicorn 启动的服务器本来可以多进程，但现在只能稳定单进程或者双进程（四进程时启动不稳定，启动 5 次失败 2 次，可能是活锁的资源竞争问题）；数据库的并发性能下降，针对异步的并发请求，我们没有成熟的高性能数据库来保障并发性请求调度的高效性了。在后续的数据库部分会给出两个数据库的 CRUD 对应关系映射与进一步说明，只要一一对应的修改，并且重新恢复学校“当代数据管理系统”的 MongoDB 账号，即可使用。

三、复现

作者致力于本项目复现的简洁性。

（一）法一：云平台容器

先进入项目的工作目录，接着“`docker build -t interknow_graph_agent .`”生成镜像，然后“`docker run -p 8000:8000 interknow_graph_agent`”实现端口映射并运行容器，最后在浏览器访问 <http://外网地址:8000/index.html>。（参见视频）

（二）法二：云平台命令行

先进入项目的工作目录，然后“`python main.py`”直接启动服务器，最后在浏览器访问 <http://外网地址:8000/index.html>。

（三）法三：本机容器

先进入项目的工作目录，接着“docker build -t interknow_graph_agent.”生成镜像，然后“docker run -p 8000:8000 interknow_graph_agent”实现端口映射并运行容器，最后在浏览器访问 <http://127.0.0.1:8000/index.html>。

（四）法四：本机命令行

先进入项目的工作目录，然后“python main.py”直接启动服务器，最后在浏览器访问 <http://127.0.0.1:8000/index.html>。

四、前端

（一）设计思路

前端采用纯静态 HTML + CSS + JS 架构，利用浏览器原生能力与 ECharts 完成数据可视化，不引入打包与构建工具，确保可直接被后端的静态资源路由按需异步下发。导航分区对应首页（index.html）、知识图谱展示（graph.html）、检索页（find.html），通过轻量脚本实现“打开即用、无感加载、实时刷新”。

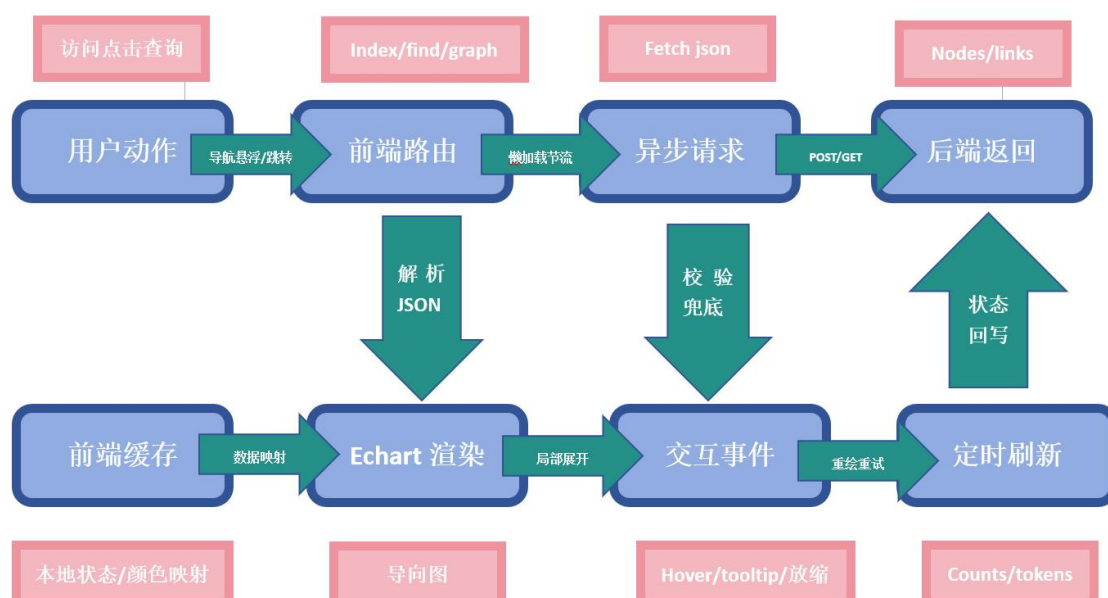


图2 前端设计思路

（二）实现方法

1. 页面分层

UI 层：templates/ 下的 index.html、graph.html、find.html 对应首页、图谱页和检索页，使用语义化标签与可访问性属性。

样式层：static/css/ 下 styles.css、navigater.css、clock_set.css 实现导航悬浮、主题色与时钟部件，利用 CSS 动画营造轻量动效。

逻辑层：static/js/ 下 app.js、counts_and_tokens.js、find.js、clock.js 负责数据获取、图谱渲染、访问量统计与时钟展示；app.js 结合 echarts.min.js 完成节点/边绘制与交互。

2. 数据获取

通过 fetch/axios-free 方式直接向后端路由发送 GET/POST 请求，遵循 nodes/links JSON 协议，收到数据后动态更新图谱和统计卡片。

3. 性能策略

懒加载：导航悬浮时才请求 graph.html，图谱数据在页面渲染后按需请求，减少首屏阻塞。

异步刷新：counts_and_tokens.js 定时轮询访问量与 tokens，采用节流与最小刷新闻隔避免高频请求。

交互裁剪：大图谱场景下应用节点剪裁（前端只渲染 N 个高权重节点）与鼠标悬浮局部展开，保障帧率。

4. 兼容性

不依赖框架，适配现代浏览器；CSS 变量集中定义主题色，便于后续换肤。

（三）实现过程：创新点与难点

我们保持了无构建链路的纯静态资源形态，能直接部署到任意静态服务器或

挂到 FastAPI 的静态目录，启动极快；导航悬浮触发页面级懒加载，配合轻量 CSS 动效，让加载几乎无感；图谱的可解释性体现在节点 hover 时展示跨学科标签与关系语义，链接颜色也区分学科来源，便于快速理解；出现空数据或校验失败时，前端会给出兜底提示和重试入口，尽量避免白屏。

至于实现过程中遇到的难点与解法，大图谱易卡顿，我们用节点裁剪并调优 ECharts 的力导参数（迭代步长、斥力、重力）减轻抖动；统计轮询与搜索都做了节流和重试退避，防止短时间的并发风暴；导航、图谱、时钟等样式独立成 CSS，JS 也按功能拆分，降低耦合与回归风险；同时统一节点配色和边透明度，保证不同规模数据下的可读性和一致性。

五、后端

后端的核心是并发性，其核心技术是协程，防止前端页面事件阻塞。后端包括了功能路由和页面路由两个板块。其中的页面路由已经在前端部分介绍，下面将介绍功能路由，即大模型相关的路由。

（一）后端中 LLM 图谱生成接口说明

后端系统中提供了面向 LLM 智能体的知识图谱生成接口，用于接收前端提交的核心概念请求，并调用智能体模块完成跨学科知识图谱的构建。该接口在系统中主要承担请求编排与结果分发的职责，而不直接参与大模型推理逻辑的实现。

在请求处理过程中，后端首先对输入参数进行基础校验，随后将概念请求传递给 LLM 智能体模块。智能体内部完成 Prompt 构造、多阶段推理以及结构化知识图谱生成，并将最终结果以标准化的节点与关系（nodes/links）结构返回后端。后端再将该结构封装为统一的响应格式，返回前端用于可视化展示。

通过将大模型调用与推理逻辑封装在智能体模块内部，后端与具体模型实现保持解耦，仅负责接口调度与数据流转，从而提高了系统整体的可维护性与扩展性。

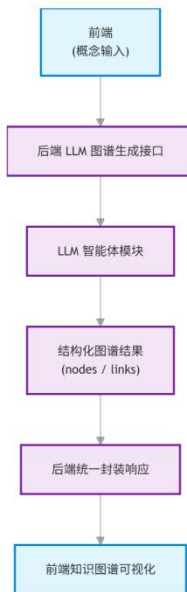


图 3 后端调用 LLM 智能体的数据流示意图

后端仅负责请求接收、接口调度与结果封装，具体的大模型推理、结构化生成与校验逻辑由 LLM 智能体模块内部完成。

（二）后端与 LLM 智能体的协同执行流程

在系统运行过程中，后端与 LLM 智能体之间通过清晰的调用与反馈机制完成知识图谱生成任务，整体执行流程如下：

首先，前端向后端提交用户输入的核心概念请求；后端接收请求后完成基础参数校验，并将请求转交给 LLM 智能体模块。智能体根据预定义的 Prompt 模板和跨学科约束，调用大模型完成多阶段推理与结构化知识图谱生成。随后，智能体对生成结果进行结构合法性与内容合理性校验（Check Layer），若校验未通过则触发修正与再生成流程；校验通过后，智能体将最终的节点与关系结构返回后端。后端对结果进行统一封装，并返回前端进行展示。

通过将复杂的推理、校验与修复逻辑集中在智能体模块内部，后端仅承担调用与结果分发职责，避免了业务逻辑与大模型细节之间的强耦合。

六、大模型（LLM Agent 工具链）

本模块实现了系统中跨学科知识发现、结构化知识图谱生成、大模型幻觉治理以及智能体策略设计等关键功能，是命题三实现的核心组成部分。

(一) 设计背景与问题分析（为什么一定要单独做 LLM 模块）

在命题三“跨学科知识图谱智能体”的任务设定中，系统表面目标是“生成一张知识图谱”，但实际难点在于：如何避免大模型只给出“看起来很合理、但没有跨学科价值”的输出。

在实验初期，如果直接调用大模型生成自然语言解释，会遇到以下问题：学科视角单一，模型往往只在最常见学科内展开（例如“熵”只谈热力学）；关系描述高度泛化，大量使用“相关”、“影响”、“联系”等低信息关系，难以支撑知识图谱；输出结果不可工程化复用，自然语言文本无法直接转化为节点—边结构；模型幻觉不可控，生成的跨学科联系缺乏可验证性，甚至逻辑错误。

基于以上问题，本项目并未将 LLM 视为“文本生成工具”，而是将其设计为一个：具备明确输入—输出规范、推理路径与校验机制的智能体 (LLM Agent)。

(二) LLM 智能体在系统中的角色定位

功能	说明
跨学科概念扩展	强制从不同学科视角挖掘关联
关系语义建模	为节点之间生成可解释关系
结构化图谱输出	输出标准 nodes/links JSON
幻觉抑制	自动识别并修正低质量输出
异常恢复	保证系统稳定性

表 3 LLM 智能体在系统中的角色定位

(三) LLM Agent 的整体架构设计

1. Agent 架构概览

LLM Agent 被设计为一个多阶段推理+校验的复合智能体，而非单次 API 调用。整体结构可抽象为以下五层：输入理解层（Input Interpretation）、跨学科推理层（Cross-domain Reasoning）、结构化生成层（Graph Construction）、幻觉校验层（Check Layer）和结果修复与重生成层（Regeneration）

2. 执行流程说明

LLM Agent 的执行流程如下：

接收用户输入的核心概念词，构造包含多重约束的 Prompt，调用大模型进行跨学科推理，提取并解析结构化输出，执行多维校验（结构 + 内容），若校验失败，进入重生成流程和输出合规的图谱结构数据。

该流程形成了一个可闭环、可恢复的智能体执行链路。

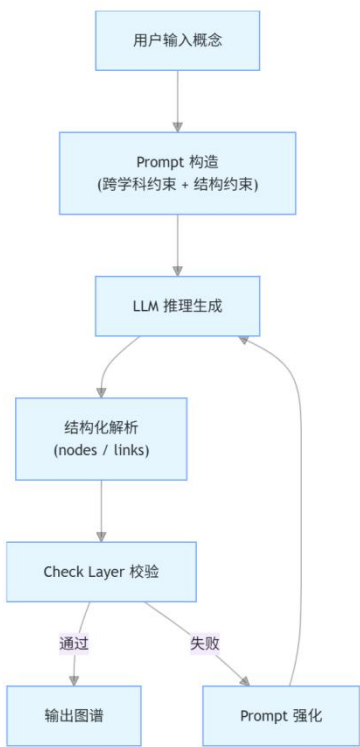


图 4 LLM Agent 多阶段推理与校验流程图

如图 4 所示，LLM Agent 的执行流程以结构化生成与幻觉校验为核心，通过再生成机制形成闭环，确保输出结果的稳定性与工程可控性。

（四）Prompt 工程的系统化设计

1. Prompt 的设计目标

在本项目中，Prompt 并非简单指令，而是承担以下工程目标：

限制模型自由发挥，强制跨学科覆盖，保证输出可解析和降低幻觉概率。

因此，Prompt 设计遵循“约束优先”原则。

2. 跨学科约束策略

为避免模型仅在单一学科内展开解释，Prompt 中明确要求：

至少涉及 3 个不同学科领域、明确标注每个概念所属学科以及优先寻找“远亲概念”而非同义词。

例如，当输入“熵”时，不仅要求涉及热力学，还必须扩展至信息论、统计学、机器学习等领域

3. 结构化输出约束

Prompt 中显式指定输出格式，要求模型生成如下结构：

```
{  
  "nodes": [...],  
  "links": [...]  
}
```

并明确禁止自由文本解释、非 JSON 格式输出和混合自然语言描述，该设计显著提高了系统的工程可控性与稳定性。

（五）幻觉校验机制（Check Layer）设计

1. 设计背景

大模型在生成跨学科关系时，极易出现这些幻觉问题：表面合理但逻辑错误的关联、关系语义过度泛化和编造不存在的学科联系。为此，本模块设计了专门的幻觉校验层（Check Layer）。

2. 结构级校验

自动检测这些问题：JSON 是否可被解析、节点是否存在重复 ID 以及边是否引用不存在节点。

若结构校验失败，系统不会直接返回错误，而是触发自动修复。

3. 内容级校验

通过规则与统计方式检测：学科数量是否达标、跨学科边比例是否充足和是否存在大量低信息关系。该机制有效抑制“看似复杂、实则空洞”的输出。

4. 幻觉抑制策略

系统对以下关系类型进行重点限制：“相关”、“影响”、“联系”和“有关”。当此类关系占比过高时，判定为低质量输出并触发重生成。

（六）再生成与容错机制

当模型输出不满足要求时，系统会自动分析失败原因、构造强化 Prompt 和引导模型进行“修正式生成”，该机制显著提升了系统的稳定性与用户体验，避免一次失败即终止流程。

（七）LLM 模块与整体系统的协同关系及云原生适配设计

1. 与前后端及后端模块的协同关系

在整体系统架构中，LLM 模块被设计为后端的核心智能服务组件，而非独立运行的脚本或临时功能模块。在向上层协同方面，LLM 模块向前端提供统一、标准化的知识图谱数据结构，其输出以节点—关系（nodes/links）的形式被前端直接用于跨学科知识图谱的动态渲染与交互展示，从而使前端无需感知大模型调用细节即可完成可视化功能。在向下层依赖方面，LLM 模块对大模型调用过程进行了完整封装，包括模型接口调用、Prompt 构造、结果解析、幻觉校验及异常处理等关键逻辑，其它后端模块无需直接接触大模型 API，显著降低了系统整体的耦合度。

2. 横向模块协作与工程稳定性保障

在后端内部，LLM 模块能够与路由模块、测试模块等组件形成清晰的横向协作关系。路由模块负责请求分发与参数校验，确保传入 LLM 模块的输入数据

合法且符合预期；测试模块则对 LLM 模块的输出结果进行自动化检查，用于验证知识图谱结构的完整性、跨学科覆盖情况以及输出格式的稳定性。通过这种横向协作机制，LLM 模块可以被自然地纳入整体工程体系，而不会成为难以调试和维护的“黑盒组件”，从而提升系统在实际运行过程中的可靠性与可维护性。

3. 云原生适配与可替换性设计

在云原生设计层面，LLM 模块被实现为一个具备良好自治能力的服务单元，强调可替换性与可扩展性。具体而言，模型类型可根据实际需求进行替换而无需修改上层业务逻辑；Prompt 模板采用集中化配置方式，便于后续调优与实验对比；幻觉校验与结果修复逻辑以模块化形式实现，为未来引入更复杂的校验策略预留了扩展空间。该设计体现了云原生架构中“职责单一、接口清晰、模块解耦”的核心理念，为系统的长期演进和规模化部署提供了稳定的工程基础。

七、数据库

1. 布隆过滤器

(1) 布隆过滤器简介

布隆过滤器（Bloom Filter）是一种空间高效的概率型数据结构，用于快速判断元素是否存在于集合中，其以可接受的假阳性误判为代价，避免了传统数据结构的高空间开销，广泛应用于大数据去重等场景，尤其是网页爬取。在我们云服务访问大模型的场景下，布隆过滤器可以大幅降低应用的运行成本，大幅提高缓存命中的响应速度。

(2) 核心原理与最优参数推导

布隆过滤器的核心是一个二进制位数组与一组独立哈希函数。对于预期存储元素数量 n ，目标假阳性误判率 p ，其最优二进制位数组长度 m 与最优哈希函数个数 k 的公式如下：

$$m = - \frac{n \times \ln(p)}{(\ln(2))^2}$$

$$k = \ln(2) \times \frac{m}{n}$$

其中， m 与 k 均需取正整数，采用向上取整方式处理。元素插入时，通过 k 个哈希函数将元素映射为位数组中的 k 个索引，并将对应索引位设为 1；元素查询时，若 k 个索引位均为 1，则判定元素“可能存在”（存在假阳性），若任意一位为 0，则判定元素“一定不存在”，无假阴性误判。

（3）具体实现

本项目给出轻量级布隆过滤器完整实现，核心依赖 `math` 库完成最优参数计算，依赖 `hashlib` 库实现多哈希函数映射。请直接参考代码，非常短小。

2. MongoDB 与 TinyDB

（一）为何使用文档数据库

图谱数据（节点+边）的本质是树状结构，而树状结构的本质又是一个嵌套的字典，而嵌套的字典就是一个 JSON 文件，即一个文档数据库的文档，因此使用 NoSQL 的 MongoDB 存储图谱数据最合适。

（二）临时调整

由于账号突然失效，数据库部分紧急使用 Python 的 TinyDB 来代替 MongoDB。

原本使用的是 `await asyncio.to_thread(数据库函数, 参数...)` 从而实现异步高并发并发控制，但是由于数据库账号突然不能使用，因此只能将实现效果下降：本来使用网络连接数据库，现在使用服务器内部的文件系统访问服务器本地的数据库；本来使用事务处理保障业务逻辑的原子性、正确性和并发控制，现在只能通过程序自身访问文件的文件锁来保证原子性；本来使用 `find_one` 来查询，现在只能用 `for` 遍历数据库来查询；本来使用 `update_one` 来更新数据库，现在只能使用 `for` 找到字典后做更新；本来可以构建 B 树索引，现在不能构建任何索引，只能扫描数据库；MongoDB 的 `insert_one` 和 TinyDB 的 `insert` 类似。

具体的对应代码请见附录。

八、云原生组件

（一）在云端部署应用后端

1. 找到一个可用的云

我们选用阿里云平台提供的云服务。本项目选择 ECS（Elastic Compute Service 弹性计算服务），为新手适用的轻量化云服务产品，且均有较为完备的新手引导教程、操作文档、AI 助手以及 AI 辅助的命令行。

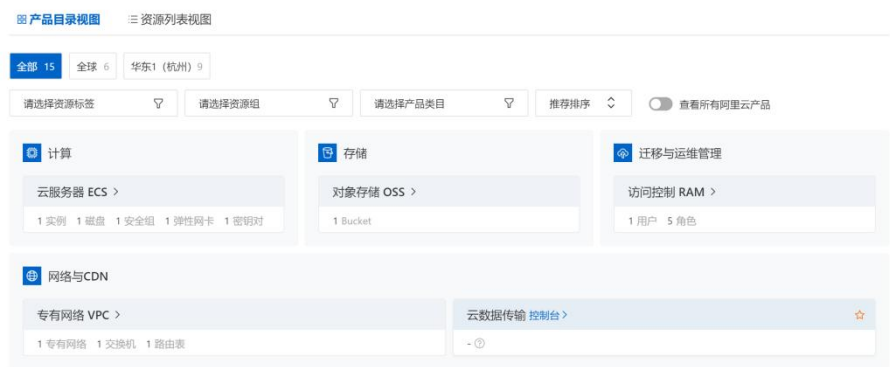


图 5 阿里云平台的 ECS

2. 业务逻辑部署在云虚拟机

（1）部署虚拟机

我们选用的云计算服务为 ECS。在购买 ECS 后，进入控制台，配置：付费类型、服务器地域、网络、实例与镜像（包括计算机指令集架构与操作系统。为了适配 Python，我们选择 Linux 系统，并设置主机用户名和密码）、扩展程序、存储、网络安全、管理模式。

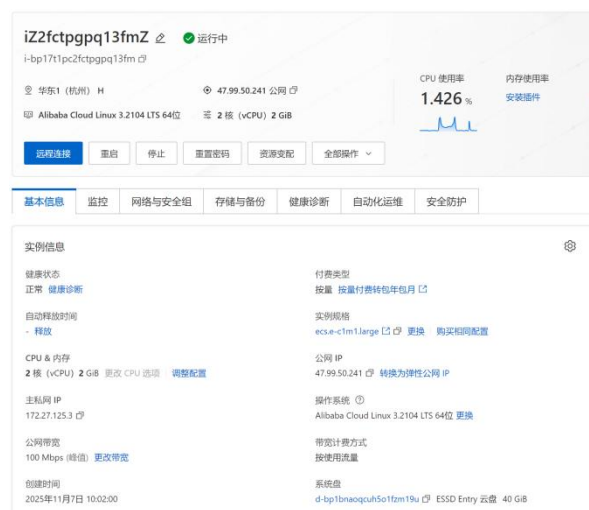


图 6 ECS 实例基本信息

(2) 连接虚拟机

创建并初始化 ECS 实例成功后，选择“Workbench 远程连接”，借助 SSH 实现远程免密连接，使得本地 PC 终端可以远程操控 ECS 云服务器。进入云服务器系统 CMD 以后，创建新的非根用户并设置密码。最后，将项目整体以压缩包的形式上传服务器端的对应文件夹，解压后运行主程序即可完成云端部署。



图 7 Workbench 远程连接方法

(3) 客户端与服务器的网络配置

在 ECS 实例的“网络与安全组”中的入方向配置防火墙（包含：优先级+协议类型+IP+port），保证任意客户端的正常访问不会被防火墙阻塞。

防火墙（安全组）规则

普通安全组
sg-bp1711pc2fcpqpln5d 更换

入方向 出方向

+ 添加入方向规则 教我配置规则

授权策略	优先级	协议类型	访问来源	访问目的(本实例)	操作
允许	1	自定义 TCP	IPv4 任何位置 (0.0.0.0/0)	端口 5500/5500	编辑 复制 删除
允许	100	Web HTTPS 流量 访问 TCP:443/443	IPv4 任何位置 (0.0.0.0/0)	端口 HTTPS(443)	编辑 复制 删除
允许	100	自定义 TCP	IPv4 任何位置 (0.0.0.0/0)	端口 5000/5000	编辑 复制 删除
允许	100	SSH 远程连接 Linux 实例 TCP:22/22	IPv4 任何位置 (0.0.0.0/0)	端口 SSH(22)	编辑 复制 删除
允许	100	RDP 远程连接 Windows 实例 TCP:3389/3389	IPv4 任何位置 (0.0.0.0/0)	端口 RDP(3389)	编辑 复制 删除
允许	100	所有 ICMP-IPv4	IPv4 任何位置 (0.0.0.0/0)	端口 全部 (-1/-1)	编辑 复制 删除

图 8 防火墙配置

在云服务器内，服务器进程的监听地址应该是“私网 IP+预定的 port”（内部监听方式），而在本地客户端进程应该访问“公网 IP+预定的 port”（外部访问方式），从而确保服务器和客户端两个进程可以正常通讯。

（2）Dockerfile 构建

Dockerfile 的逻辑非常简单，核心有三点：Python 版本号的选择、项目依赖 requirements.txt 如何配置、容器如何运行。

关于第一点，仅需进入 Docker Desktop，搜索 Python，即可查到可用的 Python 版本标签。我们使用 Python3.11。

关于第二点，requirements.txt 是通过 pip freeze 导出的，但由于我们均使用 Windows 系统，而服务器的容器却是构建在 Linux 系统上，所以是不包含 pywin32 这个包的，需要手动修改为“pywin32==306; sys_platform == "win32"”，从而避免操作系统的冲突。

关于第三点，所有应用的构建逻辑均包含在 main.py 内部，因此在容器构建完以后，直接在容器内部 python main.py 即可，没必要增加 Dockerfile 的复杂度。

```
FROM python:3.11-slim

WORKDIR /app

# System deps (build essentials for
# RUN apt-get update \
#     && apt-get install -y --no-ins
#     && rm -rf /var/lib/apt/lists/*

COPY . .

RUN pip install -r requirements.txt

EXPOSE 8000

CMD ["python", "main.py"]
```

图 9 Dockerfile 的全部内容

九、总结

本项目核心目标是构建跨学科知识图谱智能体，实现概念的跨学科关联挖掘、图谱可视化与高效查询功能。作为一个模仿实际工业生产环境的实践项目，作者额外着重考量了异步处理、并发能力、运维成本、数据缓存、历史记录缓存、容器化部署、微服务。

项目采用前后端分离架构，前端基于纯 HTML、CSS、JS 与 ECharts 实现，包含首页、知识图谱展示页、检索页三大模块，通过懒加载、异步刷新等策略保障性能与兼容性；后端以 Python 为核心，基于 FastAPI 与 Uvicorn 实现高并发异步通讯，核心功能路由聚焦大模型调用与数据处理。大模型模块设计为多阶段推理+校验的 LLM Agent，通过 Prompt 工程强化跨学科约束与结构化输出要求，搭配幻觉校验层与再生成机制，有效抑制模型幻觉，确保图谱质量。

数据存储方面，项目原计划使用 MongoDB 存储图谱数据，因账号突发失效，紧急替换为 TinyDB，虽导致并发性能有所下降，但通过程序逻辑适配保障了功能完整性；同时集成布隆过滤器，优化查询缓存命中效率，降低大模型调用成本。

部署采用云原生架构，基于阿里云 ECS 实例搭建环境，通过 Docker 容器化实现快速部署与移植，支持云平台容器、云平台命令行、本机容器、本机命令行四种复现方式，用户可通过浏览器访问对应地址即可使用。

项目整体实现了跨学科知识图谱的生成、可视化、查询与缓存等核心功能，

通过模块化设计与解耦架构保障了系统的可维护性与扩展性，同时针对部署过程中的环境问题与数据存储突发状况，提供了切实可行的解决方案。

十、参考文献

- [1] 数据科学与工程算法基础 [M]. 布隆过滤器.
- [2] 阿里云。阿里云产品技术文档 [EB/OL]. 阿里云官方文档中心, [2026.1.21]. <https://help.aliyun.com/>
- [3] MongoDB Inc. MongoDB 官方技术文档 [EB/OL]. MongoDB Documentation, [2026.1.22]. <https://www.mongodb.com/docs/>
- [4] 火山引擎。火山引擎产品技术文档 [EB/OL]. 火山引擎开发者中心, [2026.1.19]. <https://www.volcengine.com/docs/>
- [5] Ross-Ning。Docker 10 分钟快速入门 [Z/OL]. 哔哩哔哩, 2021.7.10. <https://b23.tv/stq1e59>

十一、附录

```
def simulation_other_users():
    while True:
        mongodb_client = MongoClient(
            "mongodb://ecnul0235501426:ECNU10235501426@dds-uf6800965d405e14-pub.mongodb.rds.aliyuncs.com:3717/admin",
            serverSelectionTimeoutMS=10000,
            connectTimeoutMS=10000,
        )
        db = mongodb_client['ecnul0235501426']
        users_col = db['users']
        counts = random.randint(20, 40)
        users_col.update_one(
            {
                "total_tokens": {"$exists": True},
                "total_counts": {"$exists": True},
                "$expr": {"$eq": [{"size": {"$objectToArray": "$$ROOT"}}, 3]}
            },
            {
                "$inc": {
                    "total_tokens": int(counts * random.randint(3000, 8000) / 1000),
                    "total_counts": counts
                }
            }
        )
        time.sleep(2)

def simulation_other_users():
    while True:
        db = TinyDB("db.json")
        users_col = db.table("cloud_final")

        counts = random.randint(20, 40)
        # with db_lock:
        doc = next((d for d in users_col.all() if "total_tokens" in d and "total_counts" in d), None)
        if doc:
            doc["total_tokens"] += int(counts * random.randint(3000, 8000) / 1000)
            doc["total_counts"] += counts
            users_col.update(doc, doc_ids=[doc.doc_id])
        time.sleep(2)
```

附 1 main.py 内模仿其他用户使用本应用的函数对比（上 MongoDB 下 TinyDB）

```
@app.on_event("startup")
async def startup_event():
    global mongodb_client, users_col
    while True:
        try:
            mongodb_client = MongoClient(
                "mongodb://ecnul0235501426:ECNU10235501426@dds-uf6800965d405e14-pub.mongodb.rds.aliyuncs.com:3717/admin",
                serverSelectionTimeoutMS=10000,
                connectTimeoutMS=10000,
            )
            db = mongodb_client['ecnul0235501426']
            users_col = db['users']
            await asyncio.to_thread(users_col.delete_many, {})
            await asyncio.to_thread(
                users_col.insert_one,
                {
                    "total_tokens": 50,
                    "total_counts": 250
                }
            )

            # 删除所有索引
            indexes = await asyncio.to_thread(users_col.list_indexes)
            for idx in indexes:
                if idx["name"] != "_id_":
                    await asyncio.to_thread(users_col.drop_index, idx["name"])

            # 给name建非唯一索引
            await asyncio.to_thread(users_col.create_index, "name")

            print(f"进程 {os.getpid()} 的 MongoDB 初始化成功")
            break
        except Exception as e:
            print(f"进程 {os.getpid()} 的 MongoDB 初始化失败: {str(e)}")

@app.on_event("shutdown")
async def shutdown_event():
    if mongodb_client:
        await asyncio.to_thread(mongodb_client.close)
        print(f"进程 {os.getpid()} 的 MongoDB 连接已关闭")
```

```
@app.on_event("startup")
async def startup_event():
    global users_col
    while True:
        try:
            db = TinyDB("db.json")
            users_col = db.table("cloud_final")
            users_col.truncate()
            users_col.insert({
                "total_tokens": 50,
                "total_counts": 250
            })

            print(f"进程 {os.getpid()} 的 MongoDB 初始化成功")
            break
        except Exception as e:
            print(f"进程 {os.getpid()} 的 MongoDB 初始化失败: {str(e)}")

@app.on_event("shutdown")
async def shutdown_event():
    pass
```

附 2 router.py 服务器开关机的数据库初始化函数对比（上 MongoDB 下 TinyDB）

```

print("start")
start = time.time()
if bf.contains(concept):
    doct = dict(await asyncio.to_thread(
        lambda: users_col.find_one({"concept": concept}, {"_id": 0})
    ))
    original_data = doct["graph"]
    print("Bloom Filter 热查询命中")
else:
    original_data = await asyncio.to_thread(build_knowledge_graph, concept)
    await asyncio.to_thread(users_col.insert_one, {"concept": concept, "graph": original_data})
    bf.add(concept)
end = time.time()
print("end")
print(f"{end-start}")

await asyncio.to_thread(
    users_col.update_one,
    {
        "total_tokens": {"$exists": True},
        "total_counts": {"$exists": True},
        "$expr": {"$eq": [{"size": {"$objectToArray": "$ROOT"}}, 3]}
    },
    {
        "$inc": {
            "total_tokens": int(original_data.get("tokens", 0)/1000),
            "total_counts": 1
        }
    }
)

```

```

print("start")
start = time.time()
if bf.contains(concept):
    User = Query()
    doc = users_col.get(User.concept == concept)
    original_data = doc["graph"]
    # print(original_data)
    print("Bloom Filter 热查询命中")
    if original_data==None:
        original_data = await asyncio.to_thread(build_knowledge_graph, concept)
        users_col.insert({"concept": concept, "graph": original_data})
        bf.add(concept)
else:
    original_data = await asyncio.to_thread(build_knowledge_graph, concept)
    users_col.insert({"concept": concept, "graph": original_data})
    bf.add(concept)
end = time.time()
print("end")
print(f"{end-start}")

add_tokens = int(original_data.get("tokens", 0)/1000)
add_counts = 1
doc = next((d for d in users_col.all() if "total_tokens" in d and "total_counts" in d), None)
if doc:
    doc["total_tokens"] += add_tokens
    doc["total_counts"] += add_counts
    users_col.update(doc, doc_ids=[doc.doc_id])

```

附 3 router.py 内 build_graph 的数据库操作对比（上 MongoDB 下 TinyDB）

```

await asyncio.to_thread(
    users_col.update_one,
    {
        "total_tokens": {"$exists": True},
        "total_counts": {"$exists": True},
        "$expr": {"$eq": [{"$size": {"$objectToArray": "$$ROOT"}}, 3]}
    },
    {"$inc": {"total_tokens": int(usage.get("total_tokens", 0)/1000), "total_counts": 1}}
)

```

```

doc = next((d for d in users_col.all() if "total_tokens" in d and "total_counts" in d), None)
if doc:
    doc["total_tokens"] += add_tokens
    doc["total_counts"] += add_counts
    users_col.update(doc, doc_ids=[doc.doc_id])

```

附 4 router.py 内 chat 内的数据库操作对比（上 MongoDB 下 TinyDB）

```

docs = dict(await asyncio.to_thread(
    lambda: users_col.find_one({"total_tokens": {"$exists": True}, "total_counts": {"$exists": True}}, {"_id": 0})
))

```

```

doc = next((d for d in users_col.all() if "total_tokens" in d and "total_counts" in d), None)
docs = dict(doc) if doc else {}

```

附 5 router.py 内 counts_and_tokens 的数据库操作对比(上 MongoDB 下 TinyDB)